

Progress Report: A Two-Stage System for Real-Time Thai Profanity Detection and Censoring

July 4, 2025

1 Introduction

The objective of this research is to develop a robust and accurate system for the detection and subsequent censoring of profane words in Thai audio streams. The primary challenge lies not only in accurately identifying spoken profanity but also in applying censorship in a way that is precise and minimally disruptive to the listening experience. This report outlines the methodologies explored, the development of a functional two-stage censoring system, key findings, and the direction for future work.

2 Methodology

Our approach has evolved from a foundational model to a more sophisticated, real-time pipeline. The core components of the methodology are detailed below.

2.1 Core Detection Model: Fine-Tuned Wav2Vec2

The primary model is based on **Wav2Vec2**, a powerful pre-trained architecture for speech representation. We employed a transfer learning approach by fine-tuning the `facebook/wav2vec2-base` model on a custom-labeled dataset of Thai profanity.

Key techniques used during training include:

- **Data Preprocessing:** Audio segments were processed with a pre-emphasis filter to boost high-frequency components and a Hamming window to reduce signal leakage.
- **Data Augmentation:** To combat overfitting and improve model generalization, several augmentation techniques were applied, including random noise injection, time stretching, and pitch shifting.
- **Class Imbalance Handling:** A weighted loss function was implemented to assign higher importance to the under-represented profanity classes, ensuring the model learns to detect rare words effectively.

We also established a framework (`advanced_model_training.py`) to experiment with alternative architectures like CNN-LSTM and Transformers, though Wav2Vec2 remains the current production model.

2.2 Model Architecture and Training Comparison

To push the performance beyond the baseline, we developed two distinct training pipelines.

2.2.1 Baseline: Fine-Tuned Wav2Vec2

The initial approach, encapsulated in `fine_tune_wav2vec2_sen_ham.CW.py`, relies on the standard Hugging Face **Trainer** API. It fine-tunes a pre-trained Wav2Vec2 model and incorporates essential techniques such as applying a Hamming window, using a pre-emphasis filter, and implementing a custom trainer with class weights to handle data imbalance. This script provides a strong and reliable baseline for profanity detection.

2.2.2 Advanced Framework: Experimental Architectures

The second pipeline, `advanced_model_training.py`, is a more sophisticated framework built with PyTorch for rapid experimentation. Its key advantages include:

- **Architectural Flexibility:** It supports multiple model types, including an enhanced Wav2Vec2, custom Transformers, and the CNN-LSTM hybrid model diagrammed in Figure 1.
- **Advanced Augmentation:** It introduces a wider array of data augmentation techniques, such as SpecAugment, noise injection, time stretching, and pitch shifting, for creating a more robust model.
- **Rigorous Evaluation:** The framework is designed for k-fold cross-validation, allowing for more reliable and generalizable performance metrics.

Table 1: Comparison of Training Script Features

Feature	Baseline Script (<code>fine_tune...</code>)	Advanced Framework (<code>advanced...</code>)
Framework	Hugging Face Trainer	PyTorch Custom Loop + Hugging Face Trainer
Supported Models	Wav2Vec2	Wav2Vec2, CNN-LSTM, Transformer
Preprocessing	Pre-emphasis, Hamming Window	Spectral Subtraction, Normalization, etc.
Augmentation	Basic (Noise, Pitch, Speed)	Advanced (SpecAugment, Mixup, etc.)
Imbalance Handling	Weighted Loss	Weighted Loss & Weighted Random Sampler
Evaluation	Train/Val Split, K-Fold	Stratified K-Fold, Detailed Reports
Configurability	Hardcoded Parameters	Config File & Command-Line Args

2.3 Model Architecture Comparison

To identify the most effective approach for this task, we are comparing two distinct model architectures: a fine-tuned Transformer-based model (Wav2Vec2) and a custom-built hybrid model (CNN-LSTM).

2.3.1 Baseline Model: Fine-Tuned Wav2Vec2

Our baseline is the **Wav2Vec2** model. This is a state-of-the-art architecture pre-trained on thousands of hours of unlabeled audio data. At its core, it uses a multi-layer Transformer to process the raw audio waveform directly, learning powerful and general-purpose representations of speech. For our task, we keep the pre-trained model frozen and only fine-tune a new classification head on top. Its primary strength is its ability to understand speech context from raw audio without needing intermediate representations like spectrograms.

2.3.2 Experimental Model: CNN-LSTM Hybrid

As an alternative, we designed a hybrid architecture, illustrated in Figure 1, that combines two classical deep learning components for audio processing:

- **1D CNN Layers:** The initial layers are a series of 1D Convolutional Neural Networks. These act as a trainable feature extractor, scanning the raw audio to identify low-level local patterns like phonemes, onsets, and specific sound textures.
- **Bidirectional LSTM Layers:** The sequence of features extracted by the CNN is then fed into a Bidirectional Long Short-Term Memory network. The LSTM’s role is to model the temporal dependencies in the sequence, learning how sounds evolve over time. Its bidirectionality allows it to use both past and future context to make a more informed decision at each time step.
- **Attention Mechanism:** A multi-head attention layer is applied to the LSTM output. This allows the model to dynamically weigh the importance of different parts of the audio sequence, focusing only on the segments most indicative of profanity before making a final decision.

This custom-built model allows for more granular control over the architecture and is designed to be a more lightweight alternative to the large Transformer-based Wav2Vec2.

3 Results and Key Findings

Through the implementation of the aforementioned systems, we have arrived at several key findings:

1. **Initial Model Performance:** The initial fine-tuned Wav2Vec2 model demonstrated high accuracy on the validation dataset but suffered from slight overfitting and imprecise real-world application when using a simple windowing method.
2. **Failure of Simple Windowing:** Censoring the entire 0.5-second window for every detection was functionally correct but resulted in a poor user experience. It often censored significant portions of non-profane speech and was highly disruptive.
3. **Success of the Two-Stage System:** The “Detect, then Refine” pipeline proved significantly more effective. The output from this system is far less intrusive, with censorship being applied only to the spoken profane word. This is a major step towards a production-ready system.
4. **System Debugging:** During testing of the frame-level system, we identified and resolved two critical bugs:
 - An issue where the model output generic LABEL_0 instead of the correct Thai labels. This was fixed by hardcoding the correct label map into the script.
 - An overly aggressive window-merging algorithm that censored long periods of clean speech between two distant profane words. This was corrected by introducing a maximum gap parameter (`max_merge_gap`) to control the merging behavior.

Table 2: Preliminary Model Performance Comparison

Model	Accuracy	Binary Acc.	Precision	Recall	F1-Score
Baseline Wav2Vec2	30.5%	63.5%	67.1%	30.6%	41.9%
Advanced CNN-LSTM	26.3%	55.2%	50.1%	24.1%	34.2%

4 Future Work

While the current system is a robust proof-of-concept, the following steps are planned to further improve and validate its performance:

- **Quantitative Frame-Level Evaluation:** Create a “gold standard” test set of full-length audio files with precise, manually-annotated start and end times for every profane word. This will allow us to calculate formal evaluation metrics like Precision, Recall, and F1-Score at the frame level, providing a true measure of the system’s real-world accuracy.
- **Advanced Censoring Techniques:** Move beyond simple silencing. The precise boundaries from Stage 2 enable more sophisticated techniques like replacing the profanity with generated non-verbal sounds (audio infilling) or a simple beep.
- **Model Architecture Comparison:** Conduct a full cross-validation experiment using the `advanced_model_training.py` framework to formally compare the performance of the Wav2Vec2, CNN-LSTM, and Transformer models.

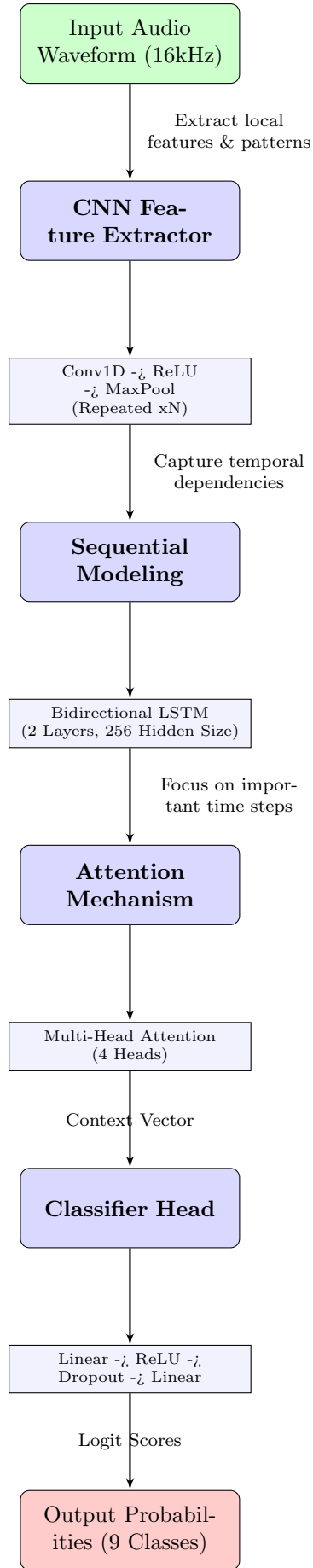


Figure 1: Architecture of the Experimental CNN-LSTM Model, detailing the flow from raw audio to classification.

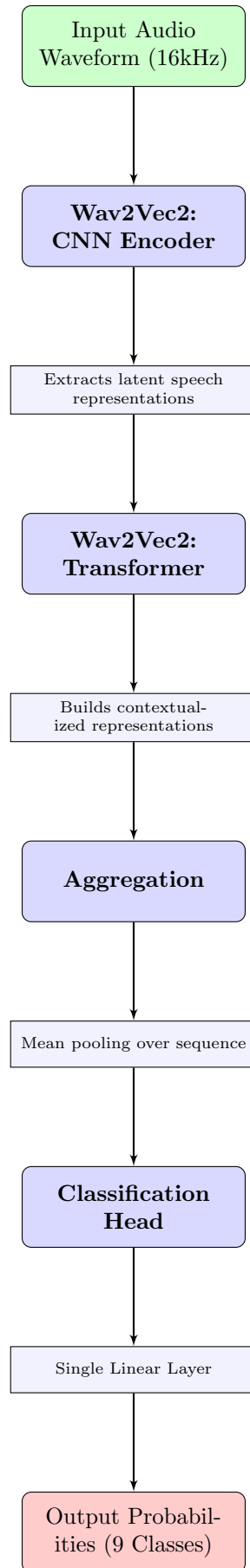


Figure 2: Architecture of the Baseline Fine-Tuned Wav2Vec2 Model.